DEVELOPER HANDBOOK

codescribe

A strictly-local, fine-tuned + retrieval-augmented + neuro-symbolic coding-assistant stack — the complete developer's guide.

Ahmad Alleboudy

Scope: everything an engineer needs to understand and build this stack — background, concept refreshers (with a deep dive on RAG and neuro-symbolic AI), architecture, the code map, the two-machine setup, the evaluation results, and the landmines that cost real debugging time.

What this is: a coding assistant for a *private codebase* — one you cannot send to a cloud model (NDA, customer contract, regulatory or security constraint). It fine-tunes a small open-weights coder on your code, serves it locally, and augments it with a retrieval index and a symbolic knowledge graph, so a modest model that *knows your codebase* beats a big model that has never seen it.

This is the teachable, source-agnostic reference. It names no specific codebase, host, or operator — everything is described by role and by concept, so it is safe to share.

Contents

Part I — Orientation

1.1 What this stack is (and is not)

codescribe turns a small, open-weights code model into a competent assistant *for one specific private codebase* — and does the whole thing on your own hardware, with nothing leaving the machine.

You have a private codebase: tens of thousands of files, years of history, lots of project-specific patterns. You want an assistant that *actually knows your code* — names your modules correctly, follows your idioms, knows where the bodies are buried — and you cannot send any of it to a cloud model. There are three ways to give a generic LLM project-specific knowledge: prompting (doesn't scale), **RAG** (retrieve relevant snippets at query time), and **fine-tuning** (bake knowledge into the weights). This stack does **fine-tuning + RAG together**, and adds a third, sharper layer — a **neuro-symbolic** knowledge graph with a deductive reasoner.

Concretely, it is a pipeline that: (1) turns any source tree into training data; (2) fine-tunes **Qwen 2.5 Coder 7B** with QLoRA on an 8 GB laptop GPU; (3) serves the result through a local OpenAI-compatible inference server; and (4) lets you drive any coding harness against it. On top sit the two memories — a RAG index (fuzzy recall of code, docs, issues, changes) and the neuro-symbolic layer (exact, provenance-backed structural facts + reasoning).

It is **not** a general chat assistant, not a hosted product, and not trying to beat frontier models. It is a single-operator system tuned to one codebase.

1.2 The prime directive: strictly local

Every design decision bows to one constraint: **data leaving the host is a failure mode, not a preference.** This is the reason the stack exists. Concrete consequences:

- **No cloud GPUs** — training and inference run on your own hardware.
- **No cloud embedding APIs** — embeddings for RAG run locally on the same GPU.
- **No cloud vector stores** — SQLite + `sqlite-vec` is the entire vector layer.
- **No telemetry** — no Weights & Biases, no Sentry, no model-hub upload, no `POST` to anything off an explicit allowlist (the model hub for *downloads*; your own Perforce and Bugzilla servers on the LAN).
- **Loopback by default** — any HTTP component binds `127.0.0.1`; binding `0.0.0.0` needs an explicit `unsafe_bind_all=True` flag and trips static-test warnings.

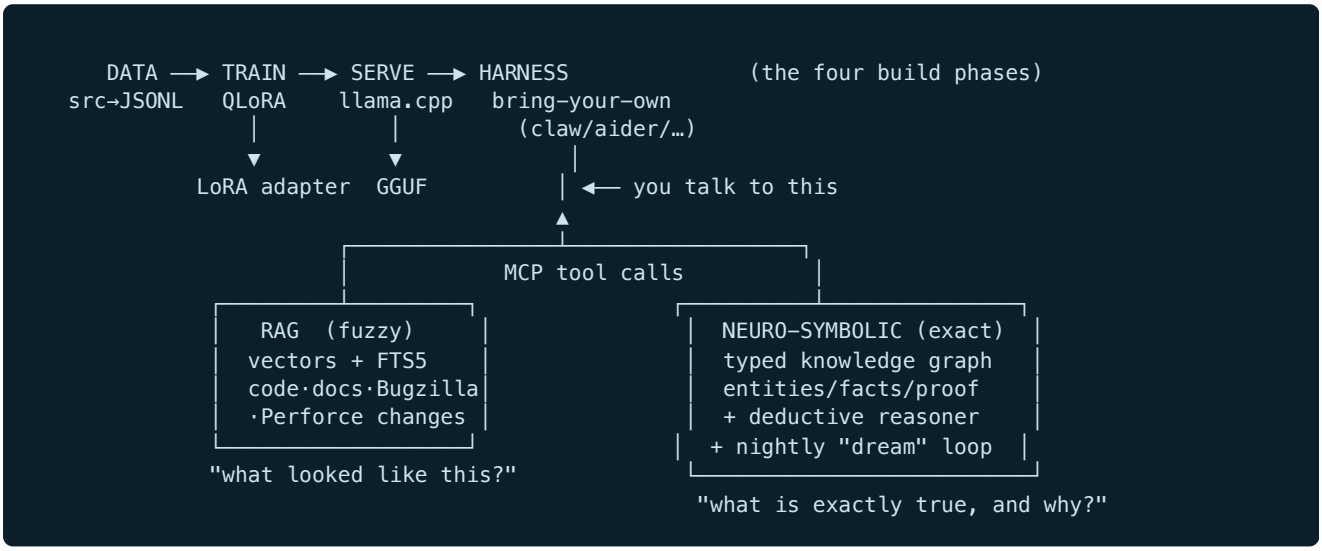
The whole system's Definition-of-Done is verified with an **egress audit**: a full session run under `strace -e trace=connect` showing *zero non-loopback `connect()` calls*. Any feature that touches the network must re-pass it.

> Rule you will not talk your way out of If a task seems to need a cloud call, an upload, or a public endpoint — **stop**. The answer is "do it locally or don't do it." This has already killed otherwise-reasonable ideas, and that is the point.

1.3 The capability ceiling

The accepted ceiling is **intern-level** assistance, and that is fine. A 7B model fine-tuned on one codebase will not reason like a frontier model. The bet: *a modest model that knows your code cold, cites real sources, and can reason over its structure* beats a bigger model that has never seen your code — for the narrow, high-frequency tasks (where is X defined, what calls Y, what breaks if I change Z, what's the convention here). Improvement comes from better base models, recipes, or local hardware — never the cloud.

1.4 The mental model: four phases + two memories



- **The four phases** depend in strict order: `data → train → serve → harness`. Each hides behind an interface so it can be swapped.
- **The two memories** are augmentations on top, both reached through **MCP tools**:
  - **RAG** — a *non-parametric, fuzzy* memory: "show me code/text that looks like this query." Fresh, broad, shallow about structure.
  - **Neuro-symbolic (NS)** — an *exact, structured* memory: "this function calls that one, defined here, and here is the proof." Precise, provenance-backed, can *reason* multi-hop.

The evaluation's headline (Part V) is that these two win on *different axes* and should be turned on for different jobs. NS is not "better RAG" — they are complementary.

Part II — Concept refreshers

Assume a strong engineer, rusty on the ML specifics. Skim what you know; the RAG (2.4) and neuro-symbolic (2.5) sections are the ones the rest of the stack leans on hardest.

2.1 Language models & the base model

An LLM predicts the next *token* given the preceding ones. Everything it "knows" is compressed into its weights — its **parametric memory**, which is (a) *frozen* at training time, (b) *lossy* (statistical tendencies, not verbatim facts — hence hallucination), and (c) *uncited*. RAG and NS exist to patch exactly these three weaknesses.

The base model is **Qwen 2.5 Coder 7B Instruct**, chosen for its strong code performance at a size that fine-tunes on 8 GB. Its tokenizer — including its **FIM** (fill-in-the-middle) tokens — is a *contract*: you cannot swap the base model without re-running the data pipeline, because the special tokens change.

2.2 Fine-tuning: LoRA → QLoRA → Unsloth

Fine-tuning continues training on your data so the weights absorb your domain. Full fine-tuning of a 7B needs far more VRAM than a laptop has; three stacked optimisations make it fit:

1. **LoRA (Low-Rank Adaptation)** — freeze the big matrices, learn small low-rank *adapters* alongside (~0.1–1% as many parameters). The output is a small, swappable adapter file, not a whole model.
2. **QLoRA (Quantized LoRA)** — additionally store the frozen base in **4-bit** (NF4), ~4× less memory; adapters still train in bf16. This is what makes a 7B trainable on 8 GB.
3. **Unsloth** — hand-optimised kernels for exactly this QLoRA path; faster and lower-memory than vanilla PEFT. Mandatory on the 8 GB card. Optimizer: `paged_adamw_8bit`.

The served model is exported to **Q4_K_M GGUF** — a 4-bit format the vendored `llama.cpp` server loads. Training samples come in three shapes (*FIM* completion, *document* language-modelling, *diff-instruction*), and splits are made **by file path, never by chunk**, so no chunk of a file leaks across train/val/test.

2.3 Model collapse & why we eval-gate

Train a model on its own synthetic output repeatedly and quality decays — variance collapses, rare patterns vanish. This is **model collapse** ("the curse of recursion"), a live risk because the neuro-symbolic layer *generates* synthetic training data. The defence is an **eval-gated retrain**, the only weight-touching phase: assemble real + capped-synthetic data, retrain, then a **beat-or-discard gate** requires the new adapter to beat the current one on a frozen, synthetic-free eval, or it is thrown away. Never promote a model that did not measurably beat its predecessor.

2.4 RAG — retrieval-augmented generation (deep dive)

RAG gives the model a second memory it can look things up in at inference time: an external, updatable index you *retrieve* from and *prepend* to the prompt. Instead of hoping the answer is

baked into the weights, you fetch relevant text and let the model read it.

```
query ──► [retrieve top-k relevant chunks] ──► prepend to prompt ──► model reads them ──► answer
                    │
          two retrieval signals, fused:
          (1) semantic  (embeddings + vector search)
          (2) lexical   (FTS5 keyword / BM25)
```

Embeddings & vector search

An **embedding** is a dense vector (here, **1024 floats**, from a local **bge-large** embedder) placing semantically similar texts near each other. Retrieve by embedding the query and finding the nearest index vectors (cosine/L2). This catches *meaning* — "authenticate a user" can match code that never says "authenticate." The vector store is **sqlite-vec**: `vec0` virtual tables holding `FLOAT[1024]`, KNN inside SQLite — the whole index is one `.db` file, no separate service.

> Fine-tune the embedder on YOUR vocabulary — the biggest retrieval lever measured Everyone fine-tunes the generator and serves the embedder stock. Measured on the reference deployment: one epoch of contrastive training on ~34k deterministically-mined pairs (path+symbol→chunk, humanized-name→definition, docstring→body; every eval-cited file excluded) moved vector-arm grounding-file retrieval **hit@1 0.573 → 0.833 (+26 points), MRR 0.665 → 0.876** — transfer, not memorization. Rules: mine pairs deterministically (no labeling), exclude eval files before training, train with the same query prefix you serve with, probe the vector arm in isolation, and remember a new embedder = a FULL vector-index rebuild. **The honest twist, twice:** composed task score moved 0 (the hybrid was no longer vector-bottlenecked), and the presumed anchorless axis then measured as a **tie** (+0.04 hit@1) — the +26 was the shape of its own training pairs, which the lexical arm already covers. Final: keep the recipe, don't ship the weights. *Component metric ≠ system metric; a fine-tune can ace the metric shaped like its own training data and add nothing.* Full arc: doc 08.

Lexical search (FTS5) and why you still need it

Embeddings blur *exact* tokens — a specific function name, an error string, a flag. **FTS5** is SQLite's full-text search (inverted index, BM25); it nails exact-identifier lookups embeddings miss. Running both and fusing them is **hybrid retrieval**.

> Three ways the lexical arm silently dies on code (all bit; docs 08/16) (1) The safe-query sanitiser joins tokens with **implicit AND** — a question-shaped query demands chunks containing *every* English word; measured: **0 BM25 rows on 100% of two eval suites' prompts**, hybrid silently vector-only. Fix: OR-retry when AND matches nothing. (2) The code FTS didn't index `file_path` — the strongest token of any file-anchored question. Fix: index it and lead the embedded text with it. (3) **Porter stemming** conflates code identifiers (`settings` → `set`); use `unicode61` with `_` as a token char for the code table, keep porter for prose tables.

RRF — fusing the two rankings

Reciprocal Rank Fusion merges the two ranked lists without comparable scores: each document scores `Σ 1/(k + rank_i)` (small constant *k*) and you re-sort. High rank by *either* signal floats a document up; high by *both* wins. Simple, robust, scale-agnostic.

What is indexed, and chunking

The index is not just code — it ingests several forms of institutional memory:

Source	Why it matters
Code files (chunked)	the actual implementation
Docs (Markdown)	design intent, READMEs
Bugzilla bugs	"why did we do X", failure history
Perforce changes / reviews	rationale, discussion, diffs

Chunking splits long files into retrievable units (functions, sections, windows) — you cannot embed a 2,000-line file as one vector meaningfully. Each chunk must keep its `file_path` *and a real line span*, and both must be *retrievable* (in the FTS columns and leading the embedded text) — chunk metadata the retrieval arms can't see doesn't exist, and without spans a model fed a chunk *cannot* produce a checkable `path:NN` citation. Injection must pack *chunk-aware* (per-chunk share of the budget, line-boundary cuts): a flat `block[:N]` cut lets chunk #1 eat the whole budget and silently drops the rest of your top-k.

> Landmine — external-content FTS5 The FTS5 tables mirror another table (`content='code_chunks'`) rather than storing their own copy. Deleting rows has a strict **order**: delete the *FTS* row **first** (while the content row still exists, so FTS recomputes the right terms to remove), *then* the vector row, *then* the content row. Wrong order leaves "ghost" terms that corrupt search. (See Part VI.)

When RAG vs fine-tuning vs tools vs NS

Use…	When you need…	Weakness
Fine-tune	fluent house style, broad structure, zero-latency recall	ages; invents specifics; can't cite
RAG	fresh, specific snippets; things newer than the fine-tune	shallow on structure; can't chain; retrieval latency
Tools (grep/etc.)	exact current file contents on demand	needs the agent to know what to look for
Neuro-symbolic	exact relationships, provenance, multi-hop reasoning	silent outside the graph; build cost

RAG's signature axis is freshness. It is measurably the *wrong* tool for structural reasoning — the multi-hop result in Part V shows it actively hurting.

2.5 Neuro-symbolic AI (deep dive)

Where the LLM is a fuzzy pattern-matcher and RAG is fuzzy lookup, the neuro-symbolic layer is an *exact*, provenance-backed knowledge graph with a *deductive reasoner*. "Neuro-symbolic" = the neural model (fuzzy, fluent) working with a symbolic system (precise, logical), each covering the other's blind spot.

Why bother, given the LLM and RAG?

1. **Be exact about structure.** "Does `A` call `B`?" has a true/false answer. A statistical model guesses; a graph *knows*.
2. **Cite real provenance.** Every symbolic fact carries the exact `file:line` it came from, so the model can relay a clickable citation.
3. **Reason multi-hop.** "Does `A` transitively call `C` through something?" needs *chaining*. LLMs are unreliable at it; RAG cannot do it at all (its chunks don't contain the connecting hop). A deductive engine chains by construction.

The knowledge graph

A typed graph stored as three core tables:

- **entities** — nodes with a `kind` (`file | symbol | module | repo | convention | decision | bug`), a canonical `name`, a `path`, and a JSON `meta` blob.
- **facts** — edges as triples *(subject) —predicate→ (object)*. Predicates: `calls`, `imports`, `defined-in`, `depends-on`, `supersedes`, `convention-applies-to`, `decision-affects`. Each carries a **confidence** and a **validity window** (`valid_from` / `valid_to`).
- **provenance** — one-to-many *evidence* per fact (`source_kind`, `source_ref` like `pkg/module.py:142`); multiple sources *raise* confidence.

Two design choices matter enormously:

- **Deterministic backbone vs. fuzzy distillation.** Facts from deterministic AST/regex parsers get **confidence 1.0**; facts *distilled by an LLM* from messy interaction logs get **< 1.0**. You can always tell a proven fact from a guessed one.
- **Temporal / soft-supersession.** Facts are not hard-deleted on change; they get a `valid_to` timestamp, so the graph keeps history and you can query "what was true as of date D."

The "dream" loop — nightly consolidation

Borrowing from how sleep consolidates memory, a nightly cycle runs five stages:

```
ingest ──▶ extract ──▶ reflect ──▶ synthesize ──▶ [retrain]
(raw       (facts       (dedup,      (reasoning-     (only if the
 episodic   from code    resolve      grounded        eval gate
 capture)   + distill    conflicts,   training        passes)
            from logs)   raise conf)  samples)
```

Raw interactions land in an episodes table; extraction turns them plus fresh code into facts; reflection deduplicates and resolves conflicts (bumping confidence when sources agree, superseding when they don't); synthesis produces reasoning-grounded training data; and — only if the eval gate passes — a retrain may run. Lineage tables record which cycles fed which weights, so a regression can be bisected.

The deductive reasoner — the intellectual core

A small, dependency-free, **read-only** backward-chaining engine over the graph. It derives *new* facts by transitive closure, each with a full proof trace. The rules are kept as *data*, Datalog-shaped and guaranteed to terminate:

```
reaches(A, B) :- calls(A, B).
reaches(A, C) :- calls(A, B), reaches(B, C).      # transitive call closure
depends(A, B) :- depends-on(A, B).
depends(A, C) :- depends-on(A, B), depends(B, C).
impacts(X, Y) :- reaches(Y, X).                    # reverse: what breaks if X changes
```

- **Proof traces.** Every derivation carries its steps — each a real fact with its `fact_id` and `source_ref` — so a caller can re-verify the whole chain.
- **Cycle-safe & depth-bounded** (BFS + a `seen` set; default max depth ~8) so evaluation always terminates on a cyclic call graph.
- **Confidence = min-of-chain** — a chain touching an LLM-distilled (<1.0) fact is visibly less than 1.0; a deterministic-only chain stays 1.0.
- **Primitives:** `reaches`, `depends`, `trace_path(src, tgt)` (shortest proof), and `impacts(x)` (the reverse call-cone: everything that reaches `x` — i.e. what a change to `x` could break).

Imagination — built and proven

"Imagination" is counterfactual reasoning built on `impacts`, in two roles. As a *serve tool*, `imagine_impact("X")` answers "what breaks if I change X?" by framing the proven impact-cone as a hypothetical. As a *training-data generator*, reasoning-chain + counterfactual **grounded synthesis** turns the reasoner's derivations into training data — every example's chain is real facts or it is rejected. A real run produced **750 examples, 0 ungrounded, 4.5× more diverse than fact-echo** synth; a fine-tune trained on it (`ft_imag`) is the *first synthetic source that helps* — it beats the baseline on both eval suites (§5.4), where fact-echo synth regressed. This **closes the loop**: reasoning → training data → a better model, gated so it cannot collapse.

> The punchline (proven in Part V) On multi-hop questions, the neuro-symbolic layer is the *only* configuration that can answer at all — **39/40 vs 0/40** for the bare fine-tune. Multi-hop reasoning is a capability the graph *adds*, not a metric the fine-tune slightly improves.

2.6 MCP — how tools reach the model

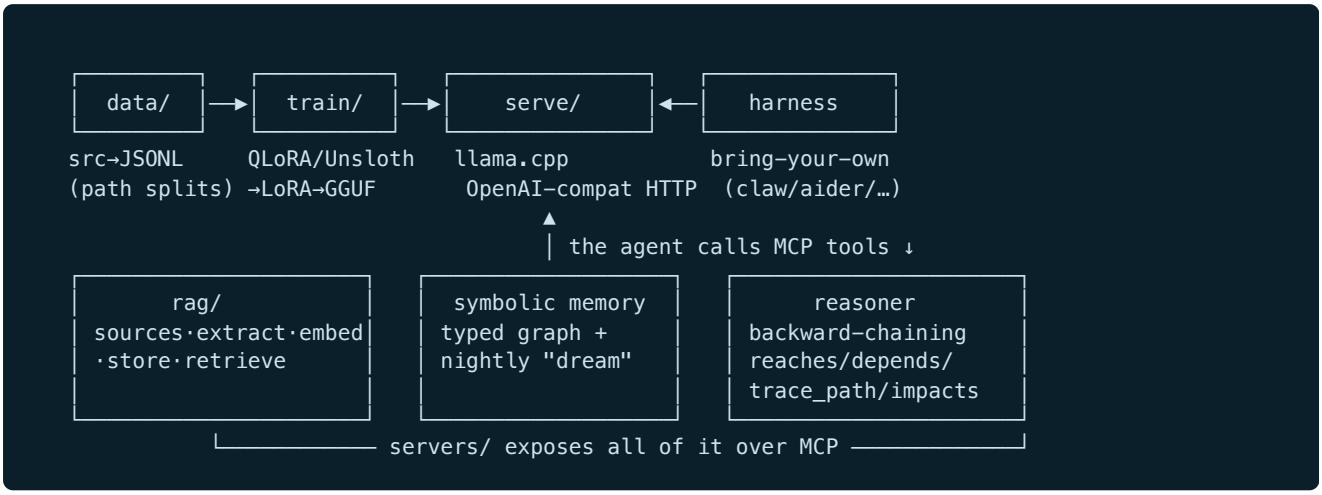
MCP (Model Context Protocol) is the standard by which the harness discovers and calls external tools. Both memories are exposed as an MCP server whose tools split cleanly:

RAG tools (fuzzy recall)	Neuro-symbolic tools (exact + reasoning)
`search_code` , `search_docs` , `search_changes` , `find_similar_bugs` , `get_change_diff`	`query_facts` , `explain_entity` , `trace_path` , `imagine_impact`

The agent decides when to call these; results are injected into its context, and a well-behaved model relays them with citations. Any OpenAI-compatible harness that speaks MCP can consume them.

Part III — Architecture & code map

3.1 The end-to-end pipeline



The four phases are locked in order and each hides behind an interface (ABC), so a phase can be reimplemented without touching the others. The lower boxes are the augmentation layers.

3.2 Package by package

The reference implementation lives under `codescribe_train/`:

Package	Purpose
`codescribe_train/data/`	Any source tree (Git working tree, Perforce workspace, or plain directory) → train/val/test JSONL. Walks files, applies a YAML config, dedups, formats (FIM / document / diff-instruction), writes deterministic **path-based** splits.
`codescribe_train/train/`	QLoRA via Unsloth over the JSONL splits; eval; Q4_K_M GGUF export. The eval-gated retrain is the only weight-touching path. Strictly local.
`codescribe_train/backends/` (serve)	Pluggable OpenAI-compatible inference layer. Default wraps vendored `llama.cpp` 's server; a remote backend exists only as a stub, never wired into a default.
`codescribe_train/harness/`	Adapter to whatever coding agent the operator picks; the harness is bring-your-own and treated as an opaque, low-trust subprocess.
`codescribe_train/rag/`	The RAG pipeline: `sources/` (code, docs, Perforce, Bugzilla), `extract/` , `embed/` , `store/` (sqlite-vec + FTS5 writer/retriever), and a CLI with `index` / `status` subcommands.
`codescribe_train/servers/`	MCP servers exposing both memories (search + facts + reasoning tools), plus a safe grep and doc lookup.
symbolic memory + *reasoner*	The typed knowledge graph, the nightly dream loop, and the deductive engine. Concepts + build plan are documented; this is the layer that wins the multi-hop eval.

3.3 The two data stores

Both stores are a single SQLite file each, built on the trainer and served on the serving box.

The knowledge graph

```
entities(id, kind, name, repo, path, meta{json}, first_seen, last_seen)
UNIQUE(kind,name,repo)
facts(id, subject→entities, predicate, object→entities | object_lit,
      confidence, extractor, created_at, valid_from, valid_to)     -- valid_to NULL =
currently true
provenance(id, fact→facts, source_kind, source_ref, excerpt, created_at)  -- evidence, many
per fact
-- plus episodes (raw interaction stream) and lineage tables (which dream cycles fed which
weights)
-- indexes: (predicate, object, valid_to) "what calls/affects X"; (subject, predicate,
valid_to) entity traversal
```

The retrieval index

```
code_chunks + code_chunk_vectors vec0[1024] + code_chunk_fts fts5(... content='code_chunks')
-- external-content!
doc_chunks  + doc_chunk_vectors  + doc_chunk_fts
bugs        + bug_vectors        + bug_fts          -- from Bugzilla
changes     + change_vectors     + change_fts       -- from Perforce (+ chunked review
discussion)
state(key, value)  -- last-indexed markers
```

> Index lifecycle Build to a temp path, verify counts + *freshness* (a staleness metric — how many indexed paths no longer exist), back up, then copy to the serving box over the LAN. The retrieval server opens the file per-session, so shipping a new index needs no restart. Shipping an index must never touch the served weights.

3.4 Configuration

Per-codebase behaviour is **data, not code**: YAML under `configs/` (separate files for the data pipeline, RAG sources + embedder + chunking, the memory/dream cycle, and the retrain gate). Dependencies install *per phase* with uv extras (`uv sync --extra data|train|backends|harness|rag`); Python 3.12; each extra pulls only its stack.

3.5 Skill selection at scale (target architecture)

A *skill* is a small named procedure a harness loads on demand — a `SKILL.md` (frontmatter `name` + `description`, body = a recipe) discovered from `.claude/skills/` in the working directory and its ancestors. Its body enters context only when invoked, so **skills cost zero standing context**: ten on the path or ten thousand is the same until one is called. "Too many skills" is therefore never a context problem — the problem at scale is *selection*: which of thousands of plugin-contributed skills to surface for a task, without showing the model a menu that would not fit.

The capability climbs three tiers as a corpus grows — **inject the menu** (≤ tens) → **lexical BM25 selector** (≤ hundreds; no embedder, no store) → **hybrid retrieval + a plugin registry + a trust/egress gate**

(thousands, many plugins) — all behind one stable seam, `select(task, k) → [descriptor]`, so climbing a tier swaps internals without touching callers. The at-scale design is six layers:

```
registry   aggregate every plugin's skills → one namespaced catalog
           (+ provenance · trust tier · egress class)
index      descriptions in their OWN sqlite-vec + FTS5 collection, isolated
           from the code index (§3.3); hybrid BM25 → vector rerank
scoping    pre-filter by active plugins · cwd/ancestors · posture
selection  top-K → scoped menu; operator /skills (primary) or a
           skill_search tool (model mode, capable models only)
loading    lazy: only the INVOKED skill's body enters context
gate       strictly-local hides network-egress skills; low-trust plugins
           quarantined; incremental, ghost-free (§16) refresh
```

Isolating the skill *index* (while optionally sharing the embedder as a library) keeps skill work in its own failure domain — it can never corrupt the code index or an evaluated retrieval matrix. This is the **main build target** for the skills layer; the full treatment, with the canonical build order and the "don't stand up a second vector RAG" reasoning, is `docs/21-skill-selection.md`.

Part IV — Infrastructure & operations

4.1 The two machine roles

The stack assumes two machines on a private LAN. Nothing reaches the cloud.

	The trainer	The serving box
Role	fine-tunes, dreams, builds indices (idle most of the time)	always-on; serves the GGUF, hosts the indices the harness reads
GPU	a modern ~8 GB card able to run the Unsloth QLoRA stack	any GPU that can serve a Q4_K_M 7B (older/cheaper is fine)
Notes	the *only* box that runs training; occasional, heavy	light, continuous; loopback bind + api-key

The division is fixed: heavy/occasional work (train, index, dream) on the trainer; light, always-on serving on the serving box. Indices are built on the trainer and copied to the serving box over the LAN. The served weights are replaced only by an explicit, eval-gated deploy.

4.2 Runbooks

Build / refresh the RAG index

```
python -m codescribe_train.rag index      # re-embed code + incremental Perforce/Bugzilla
python -m codescribe_train.rag status     # counts + STALENESS check (warns past a threshold)
# verify staleness ~0%, then copy the index file to the serving box over the LAN
```

Run the quality ablation (single-hop or multi-hop)

```
# two-process split so the embedder frees VRAM before the 7B loads (8 GB card):
python -m codescribe_train.train.ablation precompute --tasks evals/<suite>.json --out
/tmp/ctx.json …
python -m codescribe_train.train.ablation run --tasks evals/<suite>.json --contexts
/tmp/ctx.json \
    --configs ft,ft_rag,ft_ns,ft_rag_ns --report logs/ablation.json
```

Retrain (the only weight-touching path)

```
python -m codescribe_train.train.retrain --config configs/retrain.yaml
# assembles real+capped-synth → trains → BEAT-OR-DISCARD gate on a frozen synth-free eval →
deploy+rollback
```

4.3 The strictly-local posture in practice

- **Egress audit:** re-run a full session under `strace -f -e trace=connect` after wiring anything new; assert zero non-loopback connects (your own LAN box excepted, named explicitly). A stubbed embedder hides the real model-load egress — drive a *real* load.
- **Vendored trust:** third-party binaries (the inference server, the harness) are SHA-pinned, never auto-updated, and verified through the binary's own `doctor`/config-dump ("loaded 0/N" means your

config was rejected wholesale).

- **Sandbox:** in production the harness runs with no network (container `--network none`).

Part V — Evaluation: does each layer earn its place?

Evaluation answers, per axis, "should I turn this layer on for this job?" — and produced the stack's headline result.

5.1 Methodology

Evaluation is a **capability × configuration matrix**, not one number. Five configurations of the same served model, differing only in what context is injected (retrieve → prepend → complete):

Config	Fine-tune	RAG ctx	NS ctx	Isolates
base	✗ stock 7B	✗	✗	the floor
ft	✓	✗	✗	"LLM only"
ft_rag	✓	✓	✗	RAG's lift
ft_ns	✓	✗	✓	the symbolic lift
ft_rag_ns	✓	✓	✓	do they compose?

Metrics: structural accuracy (fraction of a task's expected signals present in the answer, *echo-stripped*), provenance precision (fraction of `path:NN` citations pointing to a real file with ≥NN lines), citation rate, hallucination/refusal, latency.

> Two threats you must always control (1) **Echo-inflated scoring** — if the scorer counts signals already in the prompt, every config looks better and deltas shrink; token-slice the generated answer only, and assert the fix is active at runtime. (2) **VRAM co-residency** — the embedder and the 7B don't co-fit on 8 GB, so precompute all retrieval/graph context in a process that *exits* before the model loads.

5.2 Single-hop results

56-task grounded structural suite (imports / defined-in / calls), fine-tune vs stock 7B:

config	structural	provenance	citation rate	hallucination	gen ms
base	0.537	1.000	0.018	0.000	2451
ft	0.590	0.867	0.232	0.000	2494
ft_rag	0.611	0.929	0.214	0.000	3238
ft_ns	0.623	0.941	**0.518**	0.000	2467
ft_rag_ns	0.624	**0.952**	0.375	0.000	3229

Base → FT is the real structural lift (+5 points); the augmentation deltas over FT (+0.021 / +0.033 / +0.034) are **sub-threshold ties** under the methodology's own rules (±0.018 noise floor; 5-point turn-on bar) — don't read composition into one-to-two tasks. What clears the bar: every augmentation improves provenance, and the symbolic layer's signature single-hop win is **citation rate** — handed exact

`file:line` facts, the model volunteers a checkable citation 52% of the time (vs 23% ft, 2% base), at the best precision, for ~zero extra latency. RAG adds ~740 ms.

A caveat that became a case study The first run reported ft_rag provenance at 0.73 — *below* ft — and nearly shipped "RAG hurts provenance." It was an artifact: the indexer never pruned chunks for *deleted* files, so an incremental-on-stale-base rebuild left **56% dead-path chunks**; RAG cited files that no longer existed. Fix: a prune step + a staleness detector that warns ≥5%. Corrected number: 0.929. *An ablation is only as trustworthy as the quality of the systems it compares.*

The fixed-pipeline re-run (post-audit) After the landmine-#9 retrieval fixes (file path in both arms, OR-fallback, packed blocks with line-span headers) and a clean rebuild, the RAG rows were re-measured: single-hop **ft_rag structural 0.611 → 0.700** — the +0.11 lift now *clears* the turn-on bar it previously missed — with citation rate doubled (0.446) and valid-citations-per-answer up 71%. Multi-hop ft_rag stayed **below** bare ft (0.256 vs 0.319): the reasoning deficit is intrinsic, not an implementation accident. Both §5.3 decisions survive in stronger form.

5.3 Multi-hop results — the neuro-symbolic win

40 transitive-call tasks: "does `A` transitively call `C`, through what?" The endpoints are in the prompt (they echo-strip away), so the *only* scorable signals are the **intermediate** function and a real `file:line` — exactly what a config must *chain* to produce. For the NS configs, the injected context is the reasoner's derivation.

config	structural	provenance	citation rate	named the intermediate	gen ms
ft	0.319	0.063	0.175	**0 / 40**	3176
ft_rag	0.244	0.000	0.000	1 / 40	2376
ft_ns	**0.938**	**0.813**	**0.925**	**39 / 40**	3179
ft_rag_ns	0.850	0.621	0.550	39 / 40	4182

1. **NS is the only config that can chain at all.** Bare FT names the connecting intermediate **0 / 40**; FT+NS **39 / 40**. A capability the others structurally lack, not a metric they slightly improve.
2. **RAG fails multi-hop for a deeper reason than "retrieval missed" — the audit falsified the obvious mechanism.** Probing the actual injected blocks showed the intermediate **was present in 19 of 40** of them (source file in 30/40) — yet the RAG-fed model named it **once**. Handed the same information as an explicit derivation, it names it 39/40: the binding constraint is *representation* — a small model cannot do the cross-chunk join from raw code even when the answer is in context. Plausible-adjacent chunks also *anchor* it into confident wrong "No"s, which is how FT+RAG lands below bare FT; and the "citation rate 0" was partly a metric artifact (backtick-wrapped prose citations the strict counter missed). Doc 19 §8.4 has the full anatomy.
3. **NS alone beats NS+RAG here** — the derivation is already the answer; adding chunks only distracts (and adds ~1 s). On multi-hop, turn RAG *off*: *single-query retrieve-then-read* can't chain — not because the chunks lack the hop but because the model can't assemble it; the symbolic layer is the component that does the assembly.

The decision this supports Read the matrix by column: **fine-tune for fluency and house style, RAG for freshness and single-hop lookup, the symbolic layer for provenance and multi-hop reasoning.**

Each pays for itself on its own axis — and RAG is actively wrong for reasoning. This is the empirical justification for building the neuro-symbolic layer at all.

The full lattice (post-audit) Measuring the `base_rag` / `base_ns` cells (the injected contexts are model-independent, so they reuse the ft family's exact blocks) overturned two defaults: **the symbolic layer works on the *stock* model** — every NS-carrying config clusters at 0.95–0.975 on the de-echoed suite regardless of model or retrieval (the reasoner is the capability; the model is a mouthpiece) — and **fixed retrieval helps the stock model more than the fine-tuned one on lookup** (base_rag 0.736 > ft_rag 0.700). The generic-corpus fine-tune's structural value is ~+0.05 bare and ~0 under augmentation; targeted reasoning-chain synthesis lifted +0.24 where generic corpus managed +0.11. *What you train on matters more than that you train* — budget: symbolic layer ≥ retrieval quality > generic retrains (doc 19 §8.5). Postscript: the fabrication-rate metric (a report post-processor over the graph's entity universe — store full responses and post-hoc metrics are free forever) measured the bare fine-tune inventing identifiers in **92%** of reasoning answers; the derivation halves the fabricating rate, chunks suppress frequency but fabricate heaviest when they do (doc 19, postscript).

5.4 The imagination layer: training reasoning into the weights

The matrices above measure *context injection* (retrieval / symbolic facts at inference). The imagination layer (Part II §2.5) asks whether the reasoner's derivations, turned into **grounded training data**, can teach the fine-tune *itself* to reason. `ft_imag` is a retrain on the real corpus + a capped 15% imagination synth (750 grounded examples, 0 ungrounded, bigram diversity **0.458 vs fact-echo's 0.101**), the same recipe as the baseline.

multi-hop config	task_mean	provenance	citation	named intermediate
ft (baseline)	0.319	0.063	0.175	0 / 40
ft_imag (bare)	**0.556**	**0.768**	**0.775**	**4 / 40**
ft+ns	0.938	0.813	0.925	39 / 40
ft_imag + ns	0.938	**0.974**	0.500	39 / 40

single-hop config	task_mean	provenance	citation
ft	0.590	0.867	0.232
ft_imag	**0.630**	0.789	0.268
ft+ns	0.623	0.941	0.518
ft_imag + ns	**0.693**	**0.972**	0.393

1. **The first synthetic source that *helps*.** Bare `ft_imag` beats the baseline on both suites (multi-hop 0.556 vs 0.319; single-hop 0.630 vs 0.590) and would **promote** through the beat-or-discard gate — where every fact-echo generation regressed. Reasoning-chain synth teaches structure; fact-echo did not.
2. **It bakes in *form* + *provenance*, not the exact graph.** The largest jumps are multi-hop provenance (0.063 → 0.768) and citation rate (0.175 → 0.775): the model learned to answer in explicit call-chains and cite real `file:line`. But it names the *exact* intermediate only 4/40 from weights alone — it

learned to *reason and cite*, not to memorise the graph, so the reasoner is still needed at inference for exactness.

3. `ft_imag` **+ the reasoner is the best config on both suites** — trained weights + injected derivations compose to the top numbers, and the trained-to-cite model even relays the reasoner's provenance more faithfully (0.974 vs 0.813).

> Why this is the headline contribution The imagination layer **closes the loop** — the reasoner's derivations become training data that measurably improves the fine-tune, contamination-guarded (generalization, not leakage) and gated (it cannot collapse). It is the first time any synthetic source moved the model the right way, and the empirical payoff of building the whole reasoning stack.

5.5 The final matrices — every comparison, one view

The consolidated scoreboard after the audit, the fixed retrieval pipeline, the equalized injection parameters, the full lattice, the de-echoed v2 suite, the imagination-model promotion, and the embedder arc.

config	single-hop	multi-hop	v2 positives
base	0.537	0.206	0.017
base_rag	**0.736**	0.194	0.067
base_ns	0.641	0.856	0.975
base_rag_ns	0.733	0.813	0.967
ft	0.590	0.319	0.075
ft_rag	0.700	0.256	0.075
ft_ns	0.609	**0.938**	0.950
ft_rag_ns	0.687	**0.938**	**0.975**
ft_imag *(promoted)*	0.630	0.556	—
ft_imag_ns	**0.693**	0.938	—

Reads: NS-carrying configs cluster at 0.95–0.975 on the honest suite regardless of model or retrieval (non-NS: 0.02–0.08 — a >12× capability separation); base_rag is the best single-hop cell (retrieval helps the stock model more; the fine-tune is slightly negative under every augmentation); retrieval lands below the bare model on multi-hop for both models (the hop was IN the block 19/40, named 1/40 — anchoring is intrinsic); only targeted reasoning-chain training moved weights (+0.24 vs +0.11) — and that model passed the gate (0.6304 vs 0.5896) and now serves, live-verified. Embedder arc: +26 hit@1 on its own training-pair shape → composed wash → anchorless tie → keep the recipe, don't ship. Fabrication (post-processed retroactively): bare ft invents identifiers in 92% of reasoning answers; the derivation halves it. Full tables + comparability notes: doc 19 §8.6.

5.6 Teaching the model to drive the tool loop — and what it costs

Live agentic use exposed what no offline suite measured: the fine-tune can *aim* one tool call but cannot *drive the loop* — it never reads a result, never advances, never reaches the edit tool. We trained on whole grounded trajectories (task → call → REAL result → next call → REAL result → grounded answer/edit), mined deterministically from the graph, the tree, and git history: 447 rows in three classes (locate-and-report, graph-query-then-read, commit-replay edit on the *parent-state* file), every row within the 1,024-token window, tool names byte-identical to the harness's live surface. A new 88-task held-out suite (per-file hash holdout) scores two stages with the production parser: **first_call** (right tool + key argument) and **advance** (given the call and its real result — advance, or re-issue the same call?).

model	structural (gate)	first_call	advance
champion (serving)	**0.630**	0.068	0.216
trajectories, 2 ep	0.610	**0.920**	0.943
trajectories, 3 ep	0.573	0.886	**0.977**

Reads: the trajectories teach the loop decisively (13.5× first-call, 4.5× advance, on never-trained files — and the champion's 7%/22% baseline finally quantifies the live diagnosis); **both candidates were DISCARDED by the structural hard guard** — the serving model never changed, which is the beat-or-discard gate doing its job twice; the rescue hypothesis ("2 epochs under-trained the real corpus") died on contact — 3 epochs made structure *worse*, so trajectory exposure itself trades against structural knowledge, monotonically; and the by-class table saved the story — the third epoch bought the *hardest* class (commit-replay edit 0.00/0.20 → 0.60/0.60, n=5) and paid in exactly the coin the gate protects. The loop is cheap to teach; the edit is expensive.

phase	round 1 (2 ep)	round 2 (3 ep)
trajectory generation (CPU)	2:30	reused
champion tool-use baseline	~13:50	reused
smoke gate + full train	~4:20 + 4:10:10	4:57 + 6:14:37
structural + tool-use evals	~16:00	~12:30
end-to-end	**≈4 h 44 m**	**≈6 h 32 m**

House rule adopted with this experiment: **results carry the wall-clock that bought them** — per phase, parsed from the run log's timestamped markers — so a 13× capability win can be weighed against the GPU-hours it cost. Budget was one run + one iteration, both spent; the champion keeps serving; the open levers (deliberately unscheduled): a lighter trajectory dose, a serve-time tool-adapter, or re-pricing the gate so "hands" count. Full write-up: doc 19 §8.7.

5.7 The offline→live gap, closed: when the model emits the tool RESULT, not the CALL

The follow-up to §5.6 rebuilt the trajectory data around the live prompt distribution — tool results in the harness's REAL wire formats, harness-shaped system prefixes, phrasing diversity, and an error-recovery class. Offline it was decisive; the live retest is where the lesson lives.

model	structural gate	first-call	advance-after-result
champion (serving)	**0.630**	0.129	0.565
trajectories-v2	0.628	**0.929**	**0.965**

Both *edit* classes scored **1.0 / 1.0** offline vs the champion's 0.0, and it missed the gate by **0.002**. Then the live retest found three failures, each root-caused and fixed: (1) **runaway** — under the real ~7k-token / ~50-tool prompt (max_tokens 64000) it generated 25,000+ tokens in one turn until truncation; a short prompt gets a clean single call (fix: clamp max_tokens at the shim); (2) **result-confusion** — it emitted the edit tool's RESULT payload (`<tool_response>{"path":…,"oldString":…,"newString":…}`) instead of CALLING the edit tool, having learned the harness's result formats too well on the unmasked loss (fix: the shim salvages the result-shape back into the validated call); (3) **tool-surface** — restrict the agent to the few coding tools to shrink the prompt toward training. With those in place the mechanical edit **landed** deterministically — a one-line constant 44→46, exact indentation, nothing else — through the real model, the real repair chain, and real tool execution. The precise fix the field test set out to make.

Two lessons outlast the stack: *unmasked SFT on tool-RESULT formats teaches the model to generate results, not call tools* — mask assistant-only or keep result payloads off the loss; and *an offline eval that doesn't reproduce the harness's live prompt shape measures a distribution the model never meets*. Honest scope: prescribed edits land; find-then-edit still guesses paths; novel-bug diagnosis stays human. Full detail: doc 19 §8.8.

5.8 The case study — a promotion, two defended discards, and the real ceiling

Assistant-only masking (§5.7's lesson, applied) produced the first trajectory candidate to PASS the promotion gate (0.636 vs 0.630; tool-use 0.96/0.99) — promoted to production, identity-verified, rollback staged. A six-case study of real micro-assignments (deterministic diff-marker judge, workspace restored per case) scores it **4/6**: every prescribed edit lands in 1-2 turns/seconds; search-then-edit and full diagnosis fail. Two follow-up retrains were DISCARDED by the gate defending its own champion (dose/displacement governs even with masking; an imitation class for the failing case moved nothing).

The instrumented anatomy of the stubborn case is the arc's cleanest finding: with the file resolved FOR it, the true region fed back on a miss, and the sandbox accepting its quoting, the model read the exact target text — **then answered with a location report instead of editing**, reverting to its dominant trained class (~64% of rows) at the finish line. A *behavioral prior*, not a tooling gap: in agent SFT, class balance is a behavioral dial — the majority class becomes the model's default exit. Full protocol + tables: doc 19 §8.9.

Part VI — Landmines: the traps that already bit

Each cost real debugging time. Internalise them before touching the matching subsystem.

1. The eval-echo scorer

The structural scorer originally counted signals present in the *echoed prompt*, not just the generated answer. Since ~40% of expected signals appear verbatim in their prompts, **every historical gate score was inflated ~4–7 points**. Fix: token-slice the answer only. Re-gate anything measured before the fix; treat old absolute scores with suspicion.

2. Editable install shadows PYTHONPATH

An editable install resolves imports to the install's target dir, not your `PYTHONPATH` or a sibling checkout — you can "fix" code and still run the old code. Always assert the fix is live (`inspect.getsource(fn)` contains your change; `module.__file__` points where you expect).

3. External-content FTS5 delete order

Delete FTS row → vector row → content row, in that order, or you corrupt search with ghost terms (see 2.4).

4. Stale index / no prune for deleted files

The indexer replaced a file's chunks in place but never dropped chunks for files *deleted* from the codebase. Rebuilding incrementally on a stale base accumulated dead-file chunks (hit 56%), silently poisoning retrieval and any eval using RAG. Fix: a prune step + a `status` staleness metric that warns ≥5%. Run `status` after every ship.

5. Nested-checkout indexing pollution — prune by construction, not by name

The source walkers indexed nested working-copy/worktree checkouts, so a large fraction of the index was duplicate. Excluding the scratch directory *by name* only postponed it: worktrees later reappeared under a different root-level name no list knew, and a clean rebuild came out **double** — half its chunks were duplicate checkout content. Durable fix: ONE shared exclude list for every walker (private per-walker copies drift), plus pruning any subdirectory that contains `.git` — a directory for clones, a *file* for worktrees. A nested checkout is duplicate content by construction, whatever it's called. Corollary: when an index size jumps on rebuild, diff its *composition* (top-dir counts old vs new) before shipping a diagnosis.

6. NS context for multi-hop must inject the derivation, not flat facts

The ablation's NS block originally dumped flat single-file facts. For a multi-hop task that forces the *model* to chain — the very thing it can't do — so FT+NS would have looked no better than FT (a false negative). Fix: inject the reasoner's `trace_path` derivation, parsed from the *prompt* (not the answer key). *General rule: for a reasoning eval, inject the reasoner's conclusion, not the raw facts it used.*

7. Auto-generated evals need adversarial filters

The first multi-hop generator produced degenerate tasks: builtin-method "endpoints" (called everywhere, defined nowhere) and test-name-telegraphed intermediates. Fix: require endpoint + intermediate to be

`defined-in` the codebase; reject telegraphed intermediates; prefer mid-frequency callees. Read your generated tasks and ask "how could this be answered the wrong way?"

8. Vendored-binary config schema

A stray key made a vendored binary reject its whole config ("loaded 0/N"), silently disabling a safety gate. When rendering config for a program you didn't write: confirm the discovery path, the schema (does it strict-reject unknown keys?), and end-to-end via the binary's own `doctor` /dump. A config read from source is necessary but not sufficient.

9. A comparative eval measures the whole pipeline — audit what each arm received

A "retrieval hurts reasoning" result survived one root-cause pass (the stale index, #4) and shipped with a plausible mechanism ("the chunks never contain the connecting hop"). A second audit *probed the actual injected context* and falsified it: the connecting answer **was present in ~half the blocks** — the model just couldn't assemble it from raw snippets (it relayed the same content 39/40 when handed as an explicit derivation). Stacked under the surviving numbers: a dead lexical arm (the sanitiser's implicit-AND returned **0 rows on 100% of prompts**), the file path invisible to both retrieval arms, a flat block cut that dropped all but one chunk, a citation metric blind to backtick-wrapped prose citations, and an all-YES suite half of whose signals could be scored by restating the question. None flipped the headline; all distorted sizes and mechanisms. *General rule: the score table shows the ranking; only probing the pipeline shows the mechanism — dump what each arm received, read the scorer against real answers, check the suite's answer distribution.* Full anatomy: `docs/19-evaluating-quality.md §8.4`.

10. Probe enum-ish columns before filtering on them — and make zero-yield self-explaining

The trajectory miner filtered graph entities on guessed kind names (`function/method/class`); the store's real kinds are `symbol/module/file` — zero rows, and every skip happened before the stats counters, so the run reported all-zeros with no explanation. Fix: `SELECT DISTINCT` first, cite the probe next to the filter, and count every pre-emit skip so a zero-yield run carries its own diagnosis.

11. Replay edits against the parent state, not today's tree

Commit-replay training rows windowed the *current* file while the edit came from a historical commit — a moved-on file can lack the old string entirely, so the row grounds a lie. Fix: window the parent-state text the miner already fetched for its uniqueness check. Every turn of a synthetic trajectory must be true relative to the same snapshot. (Same pass: recursive grep swept nested-checkout junk into results — one shared exclusion list for every walker, again.)

12. A hung ssh freezes a watcher silently

A monitor loop guarded only by `ConnectTimeout` froze mid-tick when an established session stalled — it never fired its terminal event, and its half-open sessions starved later connections: the watcher became the outage. Fix: `ServerAliveInterval/CountMax` on every scripted ssh, bounded single-shot fetches over whole-log streams, stop the old watcher before arming the next. A watcher must be strictly more reliable than what it watches.

13. Unmasked SFT on tool-RESULT formats teaches the model to emit results, not call tools

Training an agent on multi-turn trajectories with the harness's verbose tool-RESULT payloads on the loss: offline it scores beautifully; live, under the real prompt, it emits an edit-RESULT payload instead of calling the edit tool — it learned the result format so well it generates the result. Fix (bridge): a shim that salvages a result-shaped payload back into the call it meant (the real tool then validates it). Fix (cure): mask the SFT loss to assistant-authored tokens. *Rule: tool RESULTS are the environment's tokens, not the model's — never train the model to produce them.*

14. An offline eval that doesn't reproduce the live prompt shape measures a phantom

A tool-use suite scored 0.93/0.97 on short training-shaped prompts; the live harness sends a ~7k-token system prompt, ~50 tools, and max_tokens=64000, under which the same model ran away to context truncation and role-confused call↔result — failures absent from the offline suite. *Rule: an eval's prompt distribution is part of the metric; verify the top offline result on ONE real end-to-end run before trusting it.*

> Meta-lesson Prefer **evidence over assertion** everywhere: a reproduced bug over a suspected one, a verified swap over an assumed one, the binary's own report over a schema you read, a runtime assertion that your fixed code is the code running. Almost every landmine bit because a plausible-looking assumption went unchecked.

Part VII — Current state & roadmap

Built and verified

- The four phases: data → train → serve → harness. End-to-end verified by egress audit (zero non-loopback connects).
- RAG: sqlite-vec + FTS5 hybrid with RRF over code/docs/Bugzilla/Perforce; MCP tools; a staleness detector + prune; clean index shipped to the serving box. Plus the audited retrieval fixes (§2.4): file path in both arms, OR-fallback keeping lexical alive on question-shaped queries, code-identifier tokenizer, citable line spans end-to-end, chunk-aware injection packing, one shared walker with nested-checkout pruning.
- Neuro-symbolic: the graph with provenance + temporal supersession; the dream loop; the deductive reasoner (+ `refute_path` — the proven NO with search evidence); imagination tools (`trace_path`, `imagine_impact`).
- **Imagination synthesis (grounded).** The reasoner generates reasoning-chain + counterfactual training data (750 examples, 0 ungrounded, 4.5× the diversity of fact-echo). The fine-tune trained on it (`ft_imag`) is the first synthetic source that *lifts* the model on both eval suites (§5.4) — this closes the loop.
- Eval-gated retrain with anti-collapse gates; a first real promotion of a fine-tune over the prior one.
- Evaluation: the ablation harness + methodology + single-hop, multi-hop, and imagination-retrain results.
- **Tool-trajectory training + a tool-use eval.** 447 grounded multi-turn trajectories (deterministic miner) + an 88-task held-out suite scoring first-call and advance-after-results. The loop was taught 13×/4.5×; both candidates were gate-DISCARDED on structure (0.610 / 0.573 vs 0.630) — the champion kept serving, and the capability-vs-structure trade-off is the finding (§5.6). Experiments now report their wall-clock cost per phase.

The next steps — growing the neuro-symbolic layer

- **Done since: the imagination-trained model shipped through the gate and serves (§5.5).** The reasoning → synth → retrain cycle is live; the newest frontier it exposed is the tool-loop mix (§5.6) — a lighter trajectory dose, a serve-time tool-adapter, or re-pricing the gate so “hands” count alongside structural knowledge.
- **Negative reasoning end-to-end.** `refute_path` (built) proves "A does *not* reach C" with search evidence — nodes explored, complete-vs-depth-cut. Next: negative training synthesis (calibrated NO answers grounded in the search evidence), judged by the suite's reasoner-proven negative tasks (built).
- **More relations = more question classes.** Each new deterministic extractor unlocks a class the reasoner answers with proof: `inherits` (hierarchies), `raises/catches` (error flow), `decorates` (routes/auth: "which endpoints skip the auth decorator?"), `reads-config` (a key's blast radius), `tests` (test ↔ subject — lets the agent pick its own verification targets).
- **Name-resolution grade on edges.** Entity matching is by bare name; two same-named functions can merge in a chain. Emit qualified names where the parser knows them, stamp each edge `scoped | name-matched`, and report the weakest resolution in a proof the way min-confidence already is.

- **Temporal reasoning as a tool.** The graph versions every fact (`valid_from/to`); a `graph_diff(since)` tool ("what changed in the call graph since the fine-tune's training cut?") turns that into the freshness axis the ablation never measured.
- **Semantic entity lookup** (embedding the graph's entities) — bridges natural language to graph names; today the symbolic layer only fires when the question names the entity exactly. The reasoner stays exact — only the *entry point* gets fuzzy.
- **Rule packs (the Datalog bolt-on).** The rules are already data; let a repo declare derived predicates in config — `unguarded(X) :- endpoint_handler(X), not decorates(X, requires_auth)` — evaluated by the same backward-chainer with the same proof traces. Domain policy as queryable logic over the code graph.
- **Consolidation growth.** Grow the single-valued predicate set so the dream detects real conflicts (a symbol's `defined-in` moving files should supersede, not coexist); add a decay policy so interaction-distilled facts never re-corroborated lose confidence toward archival — the graph forgets gossip, keeps what the code re-proves nightly.
- **Skill selection at scale (§3.5).** A lexical BM25 selector is the small-corpus tier; the six-layer target — a plugin skill registry, hybrid retrieval in an *isolated* index, cwd/plugin scoping, and a trust/egress gate — is the main build target for driving this stack on a large, multi-plugin codebase. Full design + canonical build order: `docs/21-skill-selection.md`.

Where to point yourself first

1. Read the overview and architecture concept docs (the curated entry points).
2. Skim the reasoner end-to-end — it's small and the intellectual core.
3. Run the test suite (GPU-free) to see the retrieve→inject→score loop concretely.
4. Reproduce a small ablation to watch each layer's contribution.
5. Read the lessons/landmines with full file:line context before touching a subsystem.

Appendix A — Glossary

Term	Meaning
QLoRA	Quantized Low-Rank Adaptation: fine-tune by learning small adapters over a 4-bit-quantized frozen base. Fits a 7B on 8 GB.
LoRA adapter	the small trained delta (tens of MB) added to the frozen base; swappable across generations.
GGUF / Q4_K_M	the file format llama.cpp serves / its 4-bit quantization scheme.
FIM	Fill-In-the-Middle: a training format giving the model prefix+suffix to predict the middle.
RAG	Retrieval-Augmented Generation: retrieve external text at inference and prepend it to the prompt.
Embedding	a dense vector (1024-d here, from bge-large) placing semantically similar texts near each other.
sqlite-vec / vec0	SQLite extension + virtual-table type for storing vectors and doing KNN inside SQLite.
FTS5	SQLite full-text search (inverted index, BM25, stemming); the lexical half of hybrid retrieval.
RRF	Reciprocal Rank Fusion: merge ranked lists by Σ 1/(k+rank); scale-agnostic.
Neuro-symbolic (NS)	combining the neural model with a symbolic knowledge graph + logical reasoner.
Entity / Fact / Provenance	graph node / (subject-predicate-object) edge / the evidence (source_ref) backing a fact.
Backbone fact	a fact from a deterministic AST/regex extractor (confidence 1.0), vs. an LLM-distilled fuzzy fact (<1.0).
Backward chaining	proving a goal by recursively proving its sub-goals; how the reasoner derives transitive facts.
Derivation / proof trace	a derived fact plus the ordered real facts (with file:line) that prove it.
impacts / imagine_impact	the reverse call-cone of X (everything that reaches X) = "what breaks if I change X".
Dream loop	the nightly ingest→extract→reflect→synthesize→[retrain] memory-consolidation cycle.
Model collapse	quality decay from training on model-generated data recursively; why retrain is eval-gated.
MCP	Model Context Protocol: how the agent discovers and calls tools (the memories are an MCP server).
Echo-stripping	scoring only signals the model *generated*, not ones echoed from the prompt.
Ablation	turning layers on/off to measure each one's contribution per metric.

Appendix B — Package & command index

Reference-implementation layout

```
codescribe_train/data/        source tree → JSONL splits (FIM/document/diff)
codescribe_train/train/       QLoRA/Unsloth training, eval, GGUF export, eval-gated retrain,
ablation
codescribe_train/backends/    inference backends (default: vendored llama.cpp server)
codescribe_train/harness/     bring-your-own coding-agent adapter
codescribe_train/rag/         sqlite-vec + FTS5 index: sources/extract/embed/store/retrieve,
`index`/`status` CLI
codescribe_train/servers/     the MCP servers (search + facts + reasoning tools)
codescribe_train/skills/      (target) skill registry + BM25 select(task,k) + skill_search
MCP tool  [§3.5, docs/21]
configs/                      per-codebase behaviour (data, rag, memory/dream, retrain)
evals/                        the structural + multi-hop task suites
docs/                         the concept + runbook + lessons documentation tree
```

Commands

```
uv sync --extra rag|train|data|harness|backends    per-phase deps (Python 3.12)
python -m pytest tests/ -q                          the test suite (GPU-free)
python -m codescribe_train.rag index|status         build / check the RAG index
python -m codescribe_train.train.ablation …         the quality matrix
strace -f -e trace=connect …                        the egress audit (expect zero non-
loopback)
```

This handbook is the reading-order narrative that threads the concept docs together for someone starting cold. It names no specific codebase, host, or operator — it is the teachable version of the stack. Regenerate it whenever the design moves enough that the details drift.